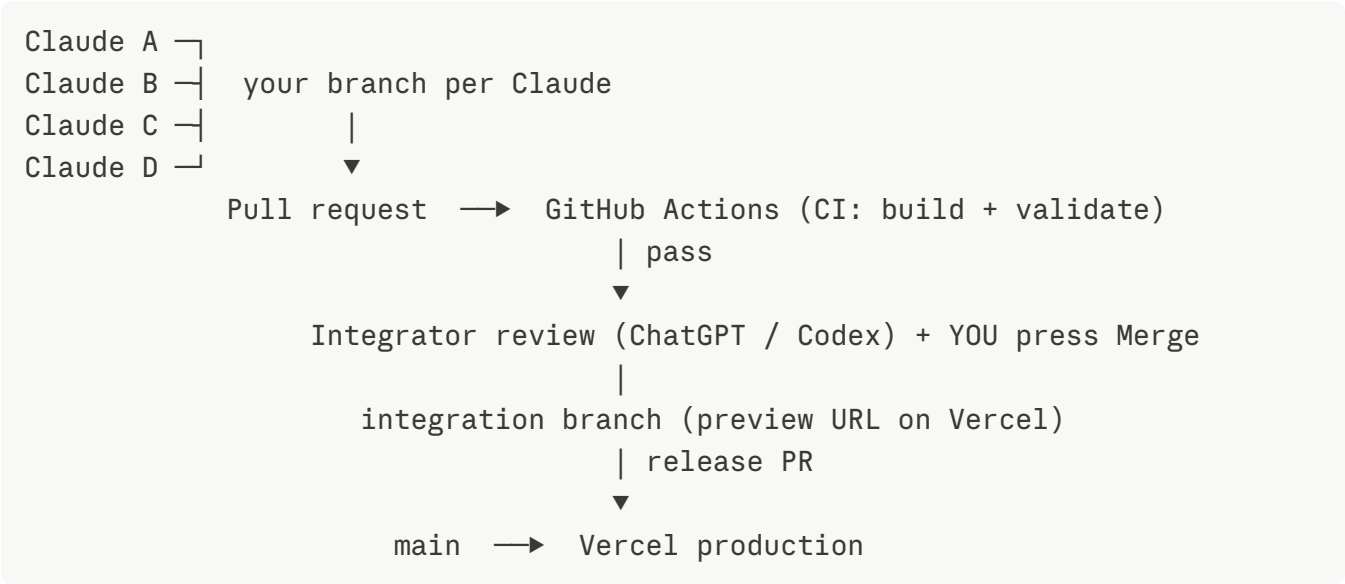
PLAYPADI Dev Manual: VS Code, GitHub, CI & Vercel, plus the 4-Claude Roadmap

Prepared for the PLAYPADI sportsbook project · November 2026

0. The big picture

You will run four Claude conversations in parallel. Each one owns a separate set of files. Their code goes into GitHub as pull requests, an "integrator" (ChatGPT/Codex connected to GitHub) reviews and merges them, GitHub Actions checks every change, and Vercel publishes whatever lands on `main`.



Golden rules

1. Nobody (human or AI) pushes straight to `main`. Everything goes through a pull request (PR).
2. Every Claude has its own lane of files. Two Claudes never edit the same file.
3. CI must be green before a merge.
4. You are the final person who clicks Merge.

---

1. Before you start

You need	Why	Where
GitHub account (turn on 2-factor login)	stores the code	github.com
Vercel account, signed in with GitHub	publishes the site	vercel.com
VS Code	your editor	code.visualstudio.com
Git	version control	git-scm.com
Node.js LTS	runs the build and checks	nodejs.org
ChatGPT with its GitHub connection	the integrator	chatgpt.com

Two things to know early

- **Vercel's free Hobby plan is for personal, non-commercial use only.** A betting platform is a commercial project, so plan for the paid Pro plan before launch. Also read Vercel's Terms and Acceptable Use for gambling-related content, and speak to a lawyer about betting licences in Nigeria. This manual is not legal advice.
- **A static site cannot take real payments or hold real balances.** Deposits, withdrawals, accounts and live odds need a secure backend later (Phase 5).

---

2. Split the project so four Claudes can work together

Your page today is one big HTML file with everything inside it. If four Claudes all edit one file, every merge becomes a conflict. So the first job is to split it into small files, each owned by one lane.

```
playpadi-web/
├─ src/
│  ├─ index.html           ← only <!-- @include --> lines (owned by D)
│  ├─ partials/
│  │  ├─ header.html       ← lane A
│  │  ├─ footer.html       ← lane A
│  │  ├─ home-main.html    ← lane B (carousel, live matches, cards)
│  │  └─ sidebars.html     ← lane C (left + right columns)
│  ├─ css/
```

```
|  |  ├─ tokens.css          ← lane D (colours, sizes, breakpoints)
|  |  ├─ base.css            ← lane D
|  |  ├─ shell.css           ← lane A
|  |  ├─ home.css            ← lane B
|  |  └─ sidebar.css         ← lane C
|  ├─ js/
|  |  ├─ shell.js            ← lane A (sticky bars, burger menu, sports ink)
|  |  ├─ home.js             ← lane B (carousel, live rows)
|  |  └─ sidebar.js          ← lane C
|  └─ assets/                ← images (logo, banners) as real files, not
base64
├─ scripts/build.js
├─ docs/CONTRACT.md
├─ .github/
|  ├─ workflows/ci.yml
|  └─ pull_request_template.md
├─ package.json
├─ vercel.json
├─ .gitignore
└─ README.md
```

Why partials? Each Claude edits only its own HTML file, and a tiny build script stitches them into one page. That removes almost all conflicts.

How to do the split: ask one Claude (lane D) with the current page attached: *"Split this single HTML file into the folder structure below. Move every base64 image into src/assets as a real file and reference it. Do not change how the page looks. Return every file in full."*

scripts/build.js

```
const fs = require("fs"), path = require("path");
const src = path.join(__dirname, "..", "src");
const out = path.join(__dirname, "..", "dist");

fs.rmSync(out, { recursive: true, force: true });
fs.mkdirSync(out, { recursive: true });

let html = fs.readFileSync(path.join(src, "index.html"), "utf8");
html = html.replace(/<!--\s*@include\s+(\S+)\s*-->/g, (_, file) =>
  fs.readFileSync(path.join(src, file), "utf8")
);
fs.writeFileSync(path.join(out, "index.html"), html);

for (const dir of ["css", "js", "assets"]) {
  const from = path.join(src, dir);
  if (fs.existsSync(from)) fs.cpSync(from, path.join(out, dir), { recursive:
true });
}
console.log("Built dist/");
```

In `src/index.html`, each partial is pulled in with a line like `<!-- @include partials/header.html -->`.

---

3. VS Code manual

3.1 Install and open the project

1. Install VS Code, then open it.
2. **File → Open Folder** and choose `playpadi-web`.
3. Open the built-in terminal with **Ctrl+`** (Mac: **Cmd+`**).

3.2 Extensions to install (Extensions icon on the left bar)

- **GitHub Pull Requests** (review and merge PRs inside VS Code)
- **GitLens** (see who changed each line)
- **Prettier – Code formatter**
- **HTMLHint** (live HTML warnings)

- **Live Server** (preview in the browser)
- **Error Lens** (shows errors on the line)
- **Path Intellisense**

3.3 Settings (`.vscode/settings.json`)

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "files.eol": "\n",
  "git.autofetch": true,
  "git.confirmSync": false
}
```

3.4 Everyday actions

Task	How
Preview the site	Run `npm run build`, then right-click `dist/index.html` → **Open with Live Server**
See what changed	**Source Control** icon (left bar) shows changed files; click one to see the before/after
Commit	type a message in the box, press the tick
Switch branch	click the branch name at the bottom-left
Fix a merge conflict	open the file, click **Resolve in Merge Editor**, pick Current / Incoming / Both, then **Complete Merge**
Paste code from a Claude	create or open the file named by the Claude, select all, paste, save

---

4. Git and GitHub manual

4.1 One-time setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Sign in to GitHub from VS Code (the **Accounts** icon, bottom-left) or run `gh auth login` if you use the GitHub CLI.

4.2 Create the repository

1. On GitHub: **New repository**, name `playpadi-web`, set it to **Private**.
2. In your project folder:

```
git init
git add .
git commit -m "chore: initial import"
git remote add origin https://github.com/YOUR-USER/playpadi-web.git
git push -u origin main
git switch -c integration
git push -u origin integration
```

Add a `.gitignore` containing `node_modules/`, `dist/`, `.env`, `.env.*` and `.DS_Store`.

4.3 Branches you will use

Branch	Purpose
`main`	what is live on the website
`integration`	where all lanes are combined and tested (gets a Vercel preview)
`lane-a/...`, `lane-b/...`, `lane-c/...`, `lane-d/...`	one branch per piece of work from each Claude

4.4 The daily loop for any lane

```
git switch integration && git pull
git switch -c lane-a/sticky-nav          # new branch for this task
# ...paste the Claude's files, run: npm run build
git add -A
git commit -m "feat(nav): sticky bar shows ball icon on scroll"
git push -u origin lane-a/sticky-nav
```

Then on GitHub click **Compare & pull request**, set the base branch to `integration`, fill in the template, and create the PR.

Commit message style: `feat(area): ...`, `fix(area): ...`, `style(area): ...`, `chore: ...`, `docs: ...`.

4.5 Protect your branches

GitHub → repository **Settings → Branches** (or **Rules → Rulesets**). For `main`:

- Require a pull request before merging
- Require approval (1)
- Require status checks to pass: select **build-and-validate** (it appears after the first CI run)
- Require branches to be up to date before merging
- Block force pushes and deletions

For `integration`, require the same status check but no approval.

4.6 Pull request template (`.github/pull_request_template.md`)

```
## Lane and what changed
- Lane: A / B / C / D
- Summary:

## Files touched (must all belong to my lane)
-

## Checked
- [ ] npm run build passes
- [ ] Looks right at 1440px, 1024px, 768px and 390px
- [ ] No edits to files owned by another lane

## Requests to other lanes
-
```

---

5. The pipeline: GitHub Actions (CI)

CI means a robot checks every pull request. If a check fails, the PR cannot be merged.

5.1 One-time project setup

```
npm init -y
npm install -D html-validate prettier
```

In `package.json`, set the scripts:

```
"scripts": {
  "build": "node scripts/build.js",
  "validate": "html-validate dist/index.html",
  "format": "prettier --write \"src/**/*.{html,css,js}\"",
  "format:check": "prettier --check \"src/**/*.{html,css,js}\""
}
```

Create `.htmlvalidate.json`. Start relaxed, then tighten later:

```
{
  "extends": ["html-validate:recommended"],
  "rules": {
    "no-inline-style": "off",
    "void-style": "off",
    "no-trailing-whitespace": "off"
  }
}
```

5.2 `.github/workflows/ci.yml`

```
name: CI
on:
  pull_request:
  push:
    branches: [main, integration]

jobs:
  build-and-validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
      - run: npm run validate
      - run: npm run format:check
      - name: Block oversized files (over 1 MB)
        run: find src -type f -size +1024k -print -exec false {} +
```

Commit `package-lock.json` too, because `npm ci` needs it. Watch a run under the repository's **Actions** tab. A green tick means pass, a red cross means open it and read the failing step.

---

6. Vercel manual

6.1 Connect the repository (recommended way)

1. Vercel → **Add New → Project** → import `playpadi-web` from GitHub.
2. **Framework Preset:** Other.
3. **Build Command:** `npm run build` · **Output Directory:** `dist`.
4. Click **Deploy**.

Optional `vercel.json` in the repo root:

```
{ "buildCommand": "npm run build", "outputDirectory": "dist", "cleanUrls": true }
```

6.2 What Vercel then does by itself

- Every pull request and every non-production branch gets its own **preview URL**. Open it on your phone to test before merging.
- Every merge to `main` goes to **Production**.
- Under **Settings → Git**, confirm the Production Branch is `main`.

Because `main` is protected and requires green CI, only checked code reaches production.

6.3 Domain, variables, rollback

- **Domain:** Settings → Domains → add your domain and follow the DNS instructions.
- **Environment variables:** Settings → Environment Variables. Never put keys in the code or the repo.
- **Rollback:** Deployments → pick the last good production deployment → **Promote / Instant Rollback**. Then fix forward with a new PR.

6.4 Optional: deploy from GitHub Actions instead

Use this only if you want Actions in full control of deployment. Turn off Vercel's automatic Git deployments for the project first, or you will deploy twice. Add repository secrets `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID` (Settings → Secrets and variables → Actions).

```
name: Deploy
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    env:
      VERCEL_ORG_ID: ${{ secrets.VERCEL_ORG_ID }}
      VERCEL_PROJECT_ID: ${{ secrets.VERCEL_PROJECT_ID }}
    steps:
      - uses: actions/checkout@v4
      - run: npm i -g vercel
      - run: vercel pull --yes --environment=production --token=${{
secrets.VERCEL_TOKEN }}
      - run: vercel build --prod --token=${{ secrets.VERCEL_TOKEN }}
      - run: vercel deploy --prebuilt --prod --token=${{ secrets.VERCEL_TOKEN
}}
```

---

7. Using four Claudes together

7.1 The four lanes

Lane	Claude	Owns (only these files)	CSS prefix	Responsible for
A: Shell	Claude A	`partials/header.html`, `partials/footer.html`, `css/shell.css`, `js/shell.js`	`hd-`, `ft-`	the three bars, sticky behaviour, ball icon, burger menu, sports bar, footer, social icons
B: Home content	Claude B	`partials/home-main.html`, `css/home.css`, `js/home.js`	`hm-`	carousel, Popular Tournaments, Live Matches, Featured Predictions, SureBet tickets, WHOT strip, loyalty banner

Lane	Claude	Owns (only these files)	CSS prefix	Responsible for
C: Sidebars & wallet	Claude C	`partials/sidebars.html`, `css/sidebar.css`, `js/sidebar.js`, later `src/pages/`	`sb-`	Register box, Betslip, Popular Competitions, Quick Links, Favourites, Top Events, Fixtures, Winners; later account, deposit, withdrawal
D: Quality & platform	Claude D	`index.html`, `css/tokens.css`, `css/base.css`, `scripts/`, `.github/`, `docs/`, `README.md`	`ui-`	design tokens, responsive and accessibility checks, performance, CI, documentation

If you use different Claude models, put the strongest on lane D (it protects everything else) and lane A (the most complex logic).

7.2 `docs/CONTRACT.md` (give this to every Claude)

This one file keeps four separate chats consistent. Fill it from your current design:

```
# PLAYPADI Contract

## Brand
Red #e11d2e · Green #16a34a (actions, odds) · Navy #0a1424 · Panel #0d1b30
Line #1c3252 · Gold #f5b027 · Lime #4ade80
Logo: PLAYPADI (nav). Alàsi logo appears beside it in the footer only.

## Layout
Body width 1000px, centred. Columns: left 180–240px | centre flexible | right
180–240px.
Navs and footer content align to the same edges as the three columns.
Breakpoints: 960 (menu collapses to burger), 899 (columns stack), 720 (mobile
bars), 599 (single column).

## Rules
- Plain HTML, CSS and JavaScript. No frameworks, no new libraries.
- Use only the colour and size variables in css/tokens.css.
```

```
- Every class starts with your lane prefix (hd-, ft-, hm-, sb-, ui-).
- Edit only the files your lane owns. Need something elsewhere? Write it
under "Requests to other lanes".
- Responsive from 390px to 1920px. Sticky header must not cover content.
- Never put secrets, keys or real user data in code.

## Removed on purpose
Hockey, Volleyball, Esports. Account/Deposit/Withdrawal pages are paused
until Phase 5.
```

7.3 The prompt to start each Claude

Paste this at the start of each of the four chats, changing the lane line:

```
You are Claude [A/B/C/D] working on the PLAYPADI sportsbook website.
Read the attached docs/CONTRACT.md and the current repo files for your lane.
Your lane: [lane name and file list from the table].
You may edit ONLY those files. Do not touch anything else.
Return format: for every changed file give the full path as a heading,
then the COMPLETE file in one code block. No partial snippets, no "rest
unchanged".
End with: (1) what you changed, (2) anything you need from other lanes.
```

When you start a new task in an old chat, re-attach the latest version of your lane's files so the Claude is not working from memory.

7.4 How code gets from a Claude to the repo

1. Claude returns full files.
2. You create a branch (section 4.4), paste each file into the exact path in VS Code, save.
3. Run `npm run build` and preview with Live Server.
4. Commit, push, open the PR into `integration`.
5. CI runs. Fix any red check (paste the error to the same Claude).

If you use **Claude Code** in the VS Code terminal for a lane, it can edit files and create the commits and branch itself. You still open the PR and merge it.

---

8. The integrator: ChatGPT connected to GitHub

8.1 What it can and cannot do (check your own account)

These tools change often, so confirm in your ChatGPT settings. At the time of writing:

- The **GitHub connector in ChatGPT chat** reads, searches and explains your repositories. It is read-only.
- **Codex** (OpenAI's coding agent, tied to your ChatGPT plan) can create branches, push changes and open pull requests through its own GitHub app, and can review a PR when you comment `@codex review`.
- Access follows what you authorise. Give it **only the `playpadi-web` repository**.

8.2 Safe setup

1. ChatGPT → Settings → connectors/apps → **GitHub** → authorise → pick `playpadi-web` only.
2. For writing and reviewing, install the **ChatGPT Codex Connector** GitHub app on the same repo, then choose that repo only.
3. Keep branch protection on. The integrator proposes and fixes. **You merge.** Never give any AI permission to bypass the protection rules.

8.3 Integrator job description

- Read each PR into `integration`.
- Check it only touches the files of its lane.
- Compare it with `docs/CONTRACT.md` (colours, class prefixes, breakpoints).
- Spot overlaps between PRs and say in what order to merge.
- When two PRs conflict, produce a fix as a new small PR (or a comment with the exact patch).
- Write a short release note for each merge.

8.4 Integrator prompts

Review a PR

```
You are the integrator for the PLAYPADI repo (playpadi-web). Review pull
request #[N]
into the integration branch. Read docs/CONTRACT.md first.
1. List every file changed and say whether it belongs to the PR's lane.
2. Flag any use of colours/sizes outside css/tokens.css and any class without
the lane prefix.
3. Check it against the other open PRs and list possible conflicts.
4. Give a verdict: approve / changes needed, with exact fixes.
```

Plan the merge

```
List all open PRs into integration. Recommend a merge order and explain why.
For each conflict, propose the smallest fix. Do not merge anything.
```

If your plan does not allow writing: the integrator gives you the patch in the chat, you paste it into the lane's branch in VS Code, commit and push. The pipeline works the same either way.

---

9. Merge protocol

Recommended merge order into `integration`

1. Lane D (tokens, base styles, CI) first, so everyone shares the same foundation.
2. Lane A (shell), because every page sits inside it.
3. Lane B (home content).
4. Lane C (sidebars).
5. After every merge, the others rebase: `git fetch origin` then `git rebase origin/integration` (or merge `integration` into your branch).

If there is a conflict

1. Do not guess. Open the file in VS Code's Merge Editor.
2. If the conflict is inside a file owned by another lane, keep that lane's version.
3. Rebuild, preview, re-run CI, then push.

Release (integration to main)

1. Open a PR from `integration` into `main`.
2. CI is green and the Vercel preview URL passes your checklist (section 10).

3. Merge with **Squash and merge** or a normal merge commit (pick one and stay consistent).
4. Vercel deploys Production automatically. Tag it: `git tag v0.1.0 && git push --tags`.

---

10. Release checklist (before merging to `main`)

- [ ] CI is green
- [ ] Preview opened on a real phone and on a laptop
- [ ] Header: 3 bars at the top; after scrolling, 2 bars stick and the ball icon shows beside Sport
- [ ] Burger menu opens and closes on mobile
- [ ] Carousel moves, arrows and dots work
- [ ] Footer logos, social icons and payment badges visible
- [ ] No broken images, no horizontal scrolling at 390px
- [ ] Lighthouse (Chrome DevTools) scores are acceptable for performance and accessibility
- [ ] No secrets in the repo

---

11. Roadmap

Phase	When	What happens	Who
0. Setup	Day 1	Accounts, tools, repo, branch protection, Vercel project connected	You
1. Split the monolith	Day 1–2	One Claude splits the single HTML into `src/` with partials and real image files; CI and build script added; everyone starts from this commit	Claude D + you
2. Sprint 1: parallel lanes	Day 3–7	A, B, C and D each work on their own branches, PRs into `integration`	4 Claudes

Phase	When	What happens	Who
3. Integration	Day 8–9	Integrator reviews, orders merges and fixes conflicts; test the `integration` preview on phone, tablet and laptop	Integrator + you
4. Release 0.1	Day 10	Release PR to `main`, checklist, production deploy, tag `v0.1.0`	You
5. Sprint 2	Week 3+	Reinstate Account, Deposit and Withdrawal pages (coins ₦ rate 100 coins = ₦1); plan a backend for sign-up, wallet, payments and live scores/odds	Claude C plus a backend plan

Weekly rhythm: Monday plan the lanes, Tuesday to Thursday build, Friday integrate and preview, then release when green.

Phase 5 reminder: real accounts, balances and PalmPay, OPay, PayPal and bank payments must run on a secure server with each provider's official integration. Do not store or process money in front-end code. Get legal advice on betting and payments licences first.

---

12. Troubleshooting

Problem	Fix
`git push` rejected	Run `git pull --rebase`, resolve conflicts, push again
"Permission denied"	Sign in again from VS Code Accounts, or run `gh auth login`
CI fails at `validate`	Open the failing step, copy the message to the lane's Claude, fix, push again
CI fails at `format:check`	Run `npm run format`, commit the changes
Vercel build fails	Check Build Command is `npm run build` and Output Directory is `dist`

Problem	Fix
Preview shows old content	Hard refresh (Ctrl+Shift+R) and check you opened the newest deployment
Image missing in production	The path is wrong or case differs (`Logo.png` is not `logo.png`)
Repo is slow or push fails on size	Keep images small (compress them), never commit videos or large exports
Two Claudes edited the same file	Keep the owner's version, move the other change into a "Request to other lane"

---

13. Cheat sheet

```
git status                          # what changed
git switch -c lane-b/new-task       # new branch
git add -A && git commit -m "feat(home): ..."
git push -u origin lane-b/new-task # first push of a branch
git fetch origin && git rebase origin/integration
git log --oneline -10               # recent commits
git restore path/to/file            # undo uncommitted edits to a file
npm run build                       # make dist/
npm run format                      # auto-format
```

14. Security basics

- Turn on 2-factor login for GitHub, Vercel and ChatGPT.
- Never paste passwords, API keys, tokens or real customer data into any Claude or ChatGPT chat.
- Secrets live only in Vercel and GitHub **Secrets**, never in files.
- Review the list of GitHub Apps with repo access every month and remove what you do not use.

Next step

Start with **Phase 0 and Phase 1**. When you have the repo created, send the current page to a Claude with the "split the monolith" request in section 2, and I can write that prompt, the starter `CONTRACT.md` and the first four lane briefs in full for you.